

SESI / SENAI AMPARO

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Unidade Curricular: SESI CE 356

Teste de Caixa Branca – Sistema de Pedido

Pietra Vitória Fernandes Lopes 3º E.M. A

Professores: Robson Bacchin e Renny Lima

Data: 30/09/2026

1. CONTEXTUALIZAÇÃO SOBRE TESTE DE CAIXA BRANCA

O teste de caixa branca é uma abordagem utilizada na avaliação de software que permite examinar o funcionamento interno do código-fonte. Ao contrário dos testes que consideram somente os dados de entrada e os resultados obtidos, essa técnica possibilita analisar a estrutura do programa, incluindo comandos, condições, decisões, caminhos de execução e o comportamento das variáveis ao longo do processamento. Dessa forma, busca-se verificar se os diferentes fluxos presentes no sistema estão sendo executados de maneira adequada e se as condições estabelecidas no código produzem os resultados esperados.

Nesta atividade, foi realizada a análise de um sistema de pedidos desenvolvido com HTML, CSS e JavaScript. A aplicação possibilita selecionar um produto, definir sua quantidade, aplicar um cupom de desconto, escolher uma modalidade de frete e, posteriormente, obter o valor total do pedido.

Catálogo de produto:

Produto	Preço	Estoque
Notebook	R\$ 3000,00	5
Mouse	R\$ 80,00	20
Teclado	R\$ 150,00	10

No código JavaScript, existem funções destinadas ao cálculo de descontos e do frete, além da função `finalizarPedido()`, responsável por validar as informações fornecidas e realizar o processamento da solicitação. Para avaliar o funcionamento do sistema, foram aplicadas principalmente as técnicas de análise de valores-limite, cobertura de decisões, cobertura de condições e rastreamento de variáveis.

A partir dos testes realizados, foram identificados seis comportamentos que apresentam possíveis falhas de lógica, os quais podem ser evidenciados pelos casos de teste descritos ao longo deste documento. As inconsistências encontradas estão relacionadas à validação da quantidade informada, à conferência do estoque, ao cálculo do desconto baseado na quantidade de produtos, ao limite estabelecido para pedidos de valor elevado e à alteração do estado de uma variável utilizada durante o processamento.

Inconsistências Detectadas na Análise do Código

Durante o processo de inspeção técnica do código-fonte, foram identificadas **6 falhas lógicas críticas** que afetam a integridade dos cálculos e a regra de negócio da aplicação:

1. **Falha na validação de quantidade mínima:** Permissão de pedidos com quantidades nulas ou negativas devido ao uso incorreto de operadores de comparação.
2. **Bloqueio indevido no limite de estoque:** Impossibilidade de adquirir a quantidade exata do último item em estoque devido ao uso de operador estritamente maior/igual ($\geq$) na checagem de indisponibilidade.
3. **Ausência de desconto por volume:** Inoperância do gatilho de desconto para compras acima de 5 unidades por conta de operador estrito ($>$), impedindo a concessão na cota limite especificada.
4. **Falha no limiar de compras de alto valor:** Falha na aplicação do desconto para pedidos com valor igual a R\$ 3.000,00 em razão do limite condicional inadequado (> 3000).
5. **Sobrescrevimento e perda de estado de status:** A mensagem de classificação do pedido (ex: "Pedido de alto valor") é sobrescrita e perdida no encerramento da rotina de cálculo.
6. **Inversão na ordem de precedência de cálculo:** A taxa de frete está sendo acrescida ao montante antes da incidência dos descontos percentuais, gerando distorção no valor final e desconto indevido sobre o frete.

2. ANÁLISE DAS ESTRUTURAS DE DECISÃO

O sistema utiliza diversas estruturas condicionais para determinar quais ações serão executadas durante o processamento do pedido. Cada condição pode direcionar o programa para um caminho diferente, de acordo com os valores informados e com as regras estabelecidas no código.

2.1 Validação da quantidade

```
if (qtd < 0) {  
    resultado.innerHTML = "<p>Quantidade inválida.</p>";  
    return;  
}
```

A condição acima é utilizada para identificar valores negativos informados na quantidade do produto. Quando isso acontece, uma mensagem de erro é apresentada e a execução da função é interrompida.

Entretanto, o campo correspondente no HTML foi definido da seguinte forma:

```
<input id="quantidade" type="number" min="1" value="1">
```

Como o valor mínimo permitido é 1, o número zero também deveria ser rejeitado. A condição atual, porém, permite que `qtd = 0` prossiga no processamento.

Para que a validação esteja de acordo com a regra definida no campo HTML, a condição pode ser alterada para:

```
if (qtd <= 0) {
```

Com essa alteração, os caminhos de execução ficam definidos da seguinte maneira:

- quantidade menor ou igual a zero => quantidade inválida;
- quantidade maior que zero => prosseguir com o processamento.

2.2 Verificação do estoque

```
if (qtd >= estoque[produtoSelecionado]) {  
    resultado.innerHTML = "<p>Quantidade indisponível em estoque.</p>";  
    return;  
}
```

Essa condição compara a quantidade solicitada com o número de unidades disponíveis no estoque. O uso do operador `>=`, entretanto, faz com que o sistema rejeite também uma quantidade que seja exatamente igual ao estoque disponível.

Por exemplo:

Estoque = 5

Quantidade solicitada = 5

A avaliação realizada pelo programa será:

`5 >= 5` => verdadeiro

Com isso, o pedido é interrompido e o sistema informa que não há estoque suficiente, mesmo existindo exatamente a quantidade solicitada.

Para permitir a compra de todas as unidades disponíveis, a comparação deve ocorrer somente quando a quantidade solicitada ultrapassar o estoque:

```
if (qtd > estoque[produtoSelecionado]) {
```

2.3 Cálculo do desconto

A função `calcularDesconto()` apresenta duas decisões responsáveis pela aplicação dos cupons promocionais:

```
if (codigo === "SENAI10") {  
    return subtotal * 0.10;  
}  
  
if (codigo === "SENAI20" && subtotal >= 1000) {  
    return subtotal * 0.20;  
}
```

Na primeira condição, o sistema verifica se o código informado é "SENAI10". Caso seja, é concedido um desconto correspondente a 10% do subtotal.

Já o cupom "SENAI20" possui uma regra adicional. Para que os 20% de desconto sejam aplicados, é necessário que o código esteja correto **e** que o subtotal seja de, no mínimo, R\$ 1.000,00.

A segunda decisão, portanto, é formada por duas condições:

```
codigo === "SENAI20"  
E  
subtotal >= 1000
```

As duas verificações precisam resultar em verdadeiro para que o desconto de 20% seja concedido.

2.4 Cálculo do frete

O cálculo do frete é realizado por meio das seguintes condições:

```
if (tipo === "retirada") {  
    return 0;  
}  
  
if (tipo === "expresso") {  
    return 60;  
}  
  
if (subtotal >= 500) {  
    return 0;  
}
```

```
return 30;
```

Nesse trecho, o valor do frete depende tanto da modalidade escolhida quanto do subtotal do pedido. Dessa forma, existem quatro possibilidades de execução:

- retirada => frete gratuito;
- expresso => frete de R\$ 60,00;
- outras modalidades com subtotal igual ou superior a R\$ 500,00 → frete gratuito;
- demais situações => frete de R\$ 30,00.

Cada condição direciona o programa para um resultado diferente, tornando esses caminhos importantes para a realização dos testes.

2.5 Desconto por quantidade

O código responsável por essa regra é:

```
if (qtd > 5) {  
    total = total - subtotal * 0.05;  
}
```

A condição determina que o desconto de 5% será aplicado somente quando a quantidade de produtos ultrapassar cinco unidades.

O valor 5 representa um limite importante para essa regra, portanto deve ser considerado nos testes de valores-limite. Se a intenção da regra for conceder o desconto a partir de cinco unidades, a condição atual não atende ao comportamento esperado.

Nesse caso, a comparação deveria ser:

```
if (qtd >= 5) {
```

Assim, tanto cinco quanto qualquer quantidade superior a esse valor receberiam o desconto.

2.6 Desconto para pedidos de alto valor

A aplicação do desconto para pedidos de valor elevado é feita por meio da seguinte condição:

```
if (total > 3000) {  
    total = total * 0.95;  
}
```

Nesse caso, o desconto de 5% é aplicado somente quando o valor do pedido ultrapassa R\$ 3.000,00.

Entretanto, em outra parte do código, o sistema utiliza uma condição diferente:

```
else if (total >= 3000) {  
    mensagem = "Pedido de alto valor.";  
}
```

Isso provoca um tratamento diferente para o valor exato de R\$ 3.000,00.

Considerando:

total = 3000

3000 > 3000 => falso

O desconto não será aplicado.

Por outro lado:

3000 >= 3000 => verdadeiro

Nesse segundo caso, o pedido será classificado como "Pedido de alto valor".

Existe, portanto, uma diferença na forma como o limite de R\$ 3.000,00 é tratado pelas duas condições, caracterizando uma inconsistência na lógica implementada.

2.7 Classificação do pedido de alto valor

Depois de realizar o cálculo do desconto, o programa utiliza novamente a variável total para definir a classificação do pedido:

```
if (total >= 3000) {  
    total = total * 0.95;  
}
```

let mensagem = "Pedido calculado com sucesso.";

```
if (total <= 0) {  
    mensagem = "Valor do pedido inválido.";   
} else if (total >= 3000) {  
    mensagem = "Pedido de alto valor.";   
}
```

O problema está no fato de que a variável total é alterada antes de ser utilizada na classificação.

Considerando um pedido com valor inicial de R\$ 3.000,00:

$3000 \geq 3000 \rightarrow$ verdadeiro

Nesse caso, o desconto é aplicado:

$3000 \times 0,95 = \text{R\$ } 2.850,00$

Depois dessa operação, a variável total passa a armazenar R\$ 2.850,00.

Quando a classificação é realizada, o programa verifica:

$2850 \geq 3000 \rightarrow$ falso

Como resultado, o sistema não classifica o pedido como de alto valor, mesmo que seu valor original tenha atingido exatamente o limite de R\$ 3.000,00.

Uma possível solução é guardar essa informação antes que total seja alterado:

```
const ehAltoValor = total >= 3000;
```

Dessa maneira, a classificação pode utilizar o valor original para verificar se o pedido atingiu o limite, enquanto total permanece disponível para representar o valor final após os descontos.

2.8 Ordem de aplicação das regras de desconto e frete

O cálculo atual do valor final segue esta estrutura:

```
let total = subtotal - desconto + valorFrete;
```

```
if (qtd >= 5) {
```

```
total = total - subtotal * 0.05;  
}
```

Nesse procedimento, o valor do frete é acrescentado ao total antes que o desconto relacionado à quantidade seja aplicado.

Apesar disso, o desconto por quantidade é calculado utilizando especificamente o subtotal dos produtos:

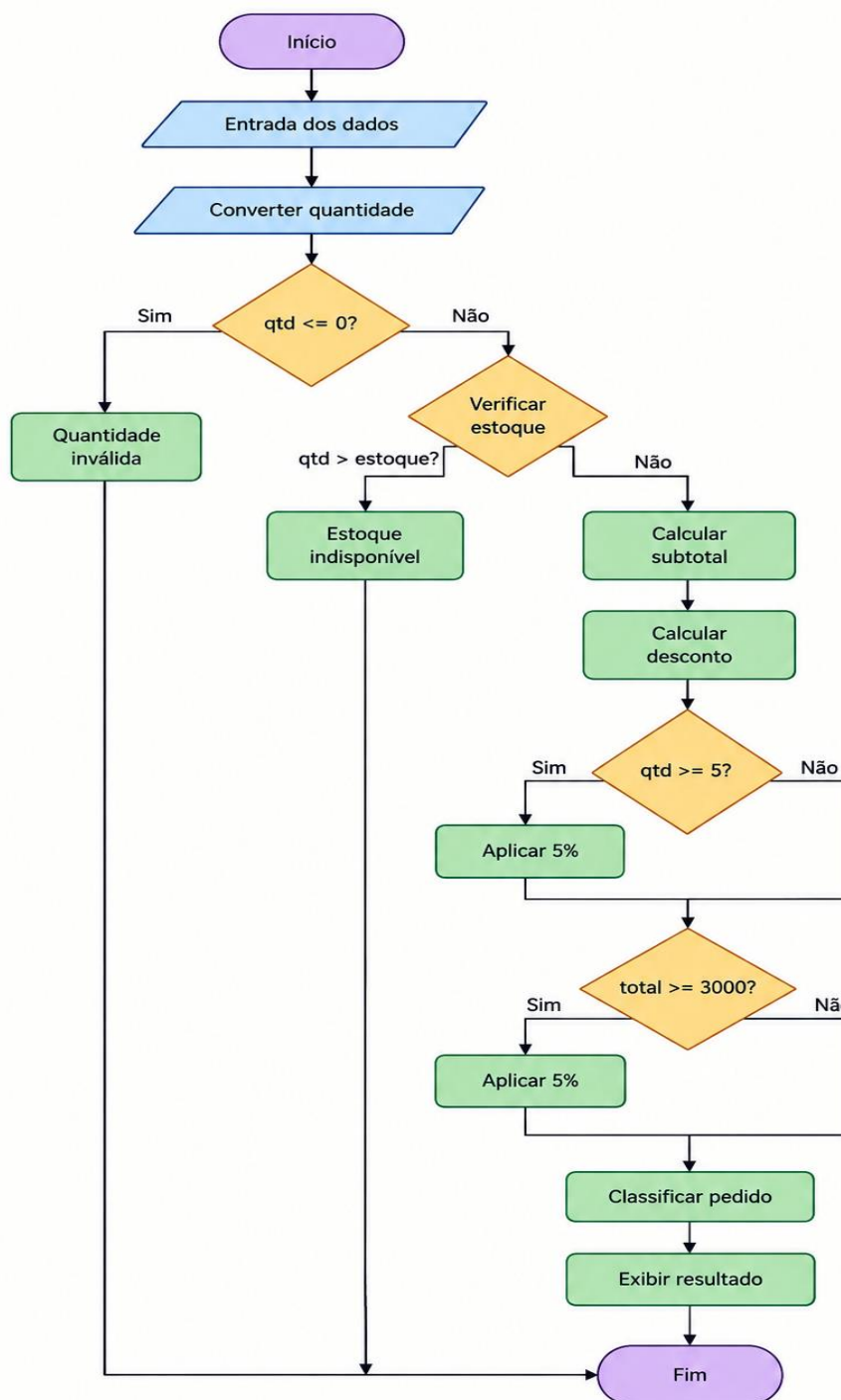
```
subtotal * 0.05
```

Para manter cada regra independente, o ideal é que os descontos referentes aos produtos sejam calculados primeiro e que o frete seja acrescentado posteriormente ao resultado.

Dessa forma, o processamento pode seguir uma ordem mais adequada: primeiro são considerados os descontos sobre os produtos, depois é calculado o valor final dos produtos e, por último, o custo do frete é adicionado.

Essa organização evita que o frete seja envolvido de maneira inadequada nas regras que dependem do valor dos produtos e torna o fluxo de cálculo mais consistente.

3. FLUXOGRAMA GERAL DO SISTEMA



4. CASOS DE TESTE

Identificação	Entrada	Condição/Caminho	Objetivo
CT01	Mouse, quantidade 0	Validação da quantidade	Verificar limite mínimo

CT02	Notebook, quantidade 5	Verificação do estoque	Verificar limite máximo disponível
CT03	Mouse, quantidade 5	Desconto por quantidade	Verificar limite de cinco unidades
CT04	Notebook, quantidade 1, retirada	Pedido de alto valor	Verificar limite de R\$ 3.000,00
CT05	Notebook, quantidade, 1	Classificação de alto valor	Verificar preservação do estado da variável
CT06	Mouse, quantidade 5, frete expresso	Ordem de cálculo	Verificar separação entre descontos e frete

5. ERRO 1 - VALIDAÇÃO DE QUANTIDADE

Nível: Fácil

Trecho do código

```
if (qtd < 0) {
    resultado.innerHTML = "<p>Quantidade inválida.</p>";
    return;
}
```

Comportamento esperado

De acordo com a configuração do campo HTML, a quantidade mínima permitida é 1 (min="1"). Dessa forma, qualquer valor igual ou inferior a zero deve ser rejeitado pelo sistema.

Dados utilizados no teste

- **Produto:** Mouse
- **Quantidade:** 0
- **Cupom:** Nenhum
- **Frete:** Normal

Caminho percorrido

Primeiramente, o valor informado pelo usuário é convertido para número:

```
qtd = Number("0")  
qtd = 0
```

Em seguida, o programa verifica a primeira condição:

```
qtd < 0  
0 < 0  
FALSO
```

Como a expressão resulta em falso, a execução não é interrompida e o sistema continua realizando as demais operações, assim, o subtotal é calculado:

$80 \times 0 = \text{R\$ } 0,00$

Resultado esperado

O comportamento correto seria interromper o processamento assim que a quantidade inválida fosse identificada e exibir a mensagem:

“Quantidade inválida.”

Resultado obtido

Na execução do teste, o sistema permite que a quantidade zero continue sendo processada. Como consequência, o subtotal calculado é:

R\$ 0,00

Erro identificado

A falha ocorre porque a condição utilizada verifica somente valores menores que zero:

```
qtd < 0
```

Com isso, o valor 0 não é considerado inválido. Essa situação contradiz a regra estabelecida pelo campo HTML, que define 1 como quantidade mínima.

Correção realizada

A condição foi modificada de:

```
if (qtd < 0) {
```

para:

```
if (qtd <= 0) {
```

Resultado após a correção

Com a nova condição e considerando:

qtd = 0

Quando o programa identifica a quantidade igual a zero, interrompe-se o processamento e apresenta:

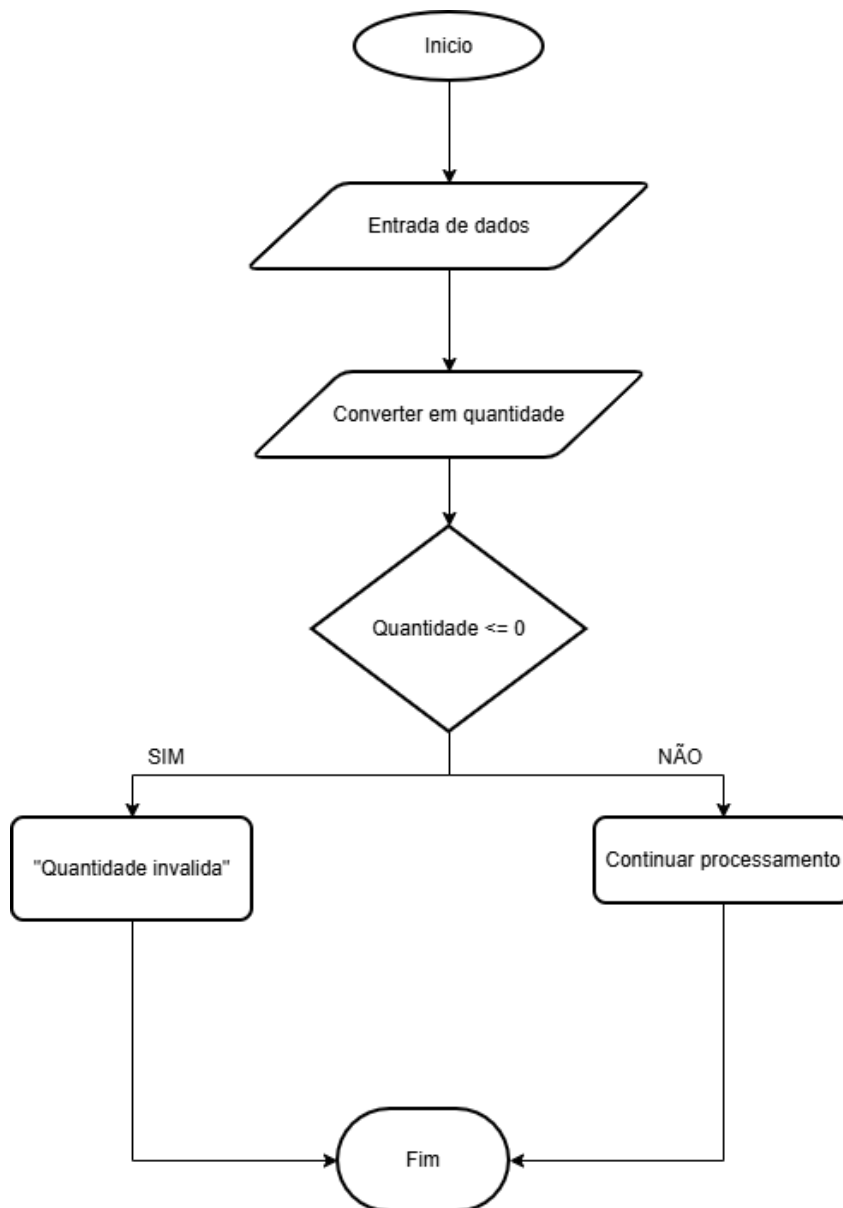
“Quantidade inválida.”

Técnica utilizada

Análise de valores-limite.

O valor zero foi utilizado por estar diretamente relacionado ao limite mínimo definido para a quantidade do produto.

Fluxograma



6. ERRO 2 — VERIFICAÇÃO DO ESTOQUE

Nível: Fácil

Trecho do código

```
if (qtd >= estoque[produtoSelecionado]) {  
    resultado.innerHTML = "<p>Quantidade indisponível em estoque.</p>";  
    return;  
}
```

}

Comportamento esperado

Quando a quantidade solicitada for exatamente igual ao número de unidades disponíveis, o sistema deve permitir o pedido. A indisponibilidade deve ocorrer somente quando a quantidade solicitada ultrapassar o estoque existente.

Dados utilizados no teste

- **Produto:** Notebook
- **Quantidade:** 5
- **Estoque disponível:** 5
- **Cupom:** Nenhum
- **Frete:** Retirada

Caminho percorrido

Inicialmente, a quantidade informada passa pela validação:

$5 < 0 \Rightarrow \text{falso}$

Como o valor não é negativo, o programa prossegue para a verificação do estoque.

Os valores utilizados são:

$qtd = 5$

$estoque = 5$

A condição existente no código é avaliada da seguinte forma:

$5 \geq 5 \Rightarrow \text{verdadeiro}$

Como o resultado é verdadeiro, a execução é interrompida e o sistema considera que não há estoque suficiente.

Resultado esperado

O processamento deveria continuar, pois existem exatamente cinco notebooks disponíveis e o usuário solicitou justamente cinco unidades.

Resultado obtido

Em vez de prosseguir com o cálculo do pedido, o sistema apresenta a mensagem:

“Quantidade indisponível em estoque.”

Erro identificado

O problema está no uso do operador `>=`. Essa comparação considera que a quantidade está indisponível tanto quando ultrapassa o estoque quanto quando é exatamente igual a ele. Porém, cinco unidades podem ser solicitadas quando existem exatamente cinco disponíveis.

Correção realizada

A condição foi modificada de:

```
if (qtd >= estoque[produtoSelecionado]) {
```

para:

```
if (qtd > estoque[produtoSelecionado]) {
```

Resultado após a correção

Com a alteração, a mesma situação passa a ser avaliada desta forma:

$5 > 5 \rightarrow \text{falso}$

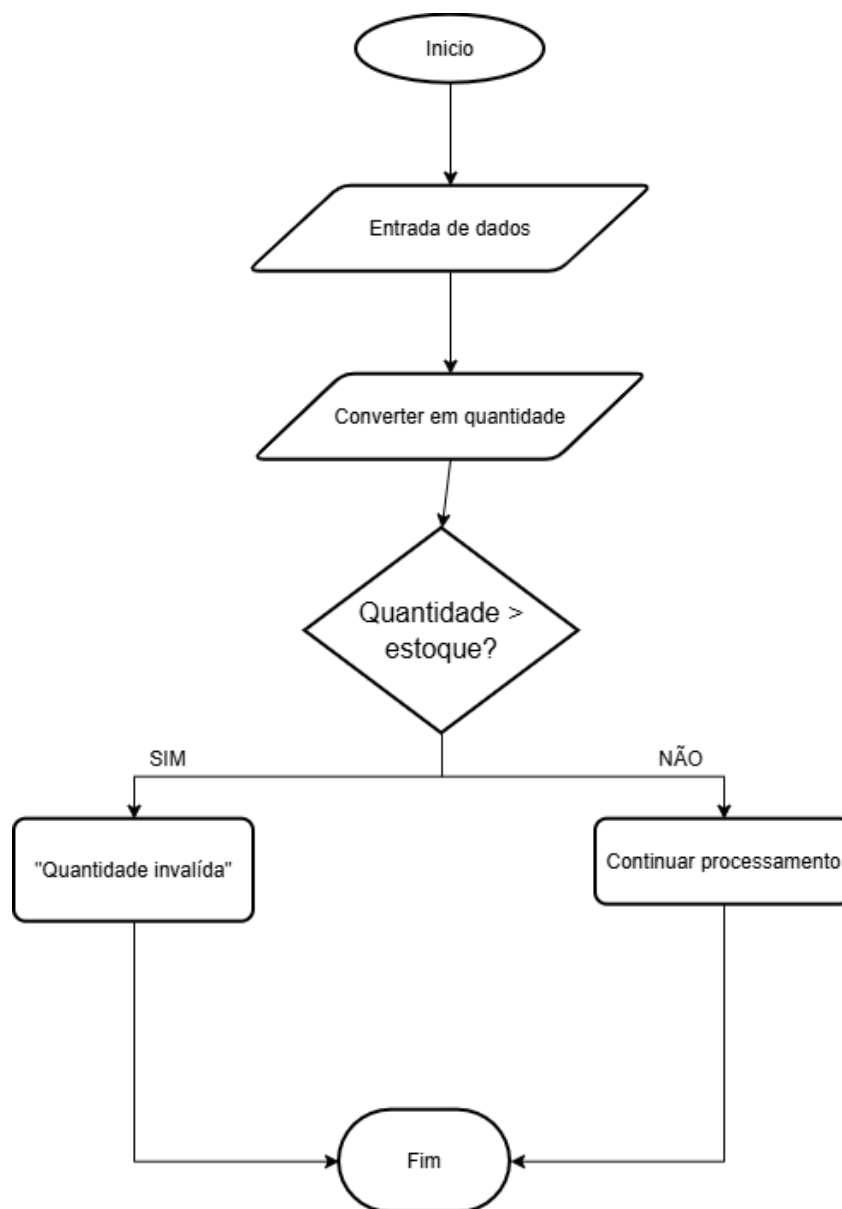
Como a condição é falsa, o programa não interrompe a execução e permite que o pedido continue sendo processado.

Técnica utilizada

Análise de valores-limite.

O teste utiliza o valor exatamente igual ao limite do estoque para verificar como o sistema se comporta nessa situação.

Fluxograma



7. ERRO 3 – DESCONTO POR QUANTIDADE

Nível: Médio

Trecho do código

```
if (qtd > 5) {  
    total = total - subtotal * 0.05;  
}
```

Comportamento esperado

De acordo com a regra estabelecida, o desconto de 5% deve ser concedido para pedidos que possuam cinco ou mais unidades. Portanto, quando a quantidade for exatamente igual a 5, o desconto também deverá ser aplicado.

Dados utilizados no teste

- **Produto:** Mouse
- **Quantidade:** 5
- **Preço unitário:** R\$ 80,00
- **Cupom:** Nenhum
- **Frete:** Normal

Caminho percorrido

A quantidade informada é:

$qtd = 5$

Na validação inicial, o sistema verifica:

$5 < 0 \Rightarrow \text{falso}$

Como a quantidade é válida, o processamento continua.

Em seguida, ocorre a verificação do estoque:

$5 \geq 20 \rightarrow \text{falso}$

Como existem 20 unidades disponíveis, o pedido pode prosseguir.

Nenhum cupom de desconto foi informado.

Para o frete normal, o valor considerado é:

R\$ 30,00

Assim, antes da aplicação do desconto por quantidade, o total é:

$400 + 30 = \text{R\$ } 430,00$

Depois disso, o programa verifica a condição responsável pelo desconto:

$5 > 5 \Rightarrow$ falso

Como a expressão resulta em falso, os 5% de desconto não são aplicados.

Resultado esperado

Como o pedido possui exatamente cinco unidades, o desconto deveria ser concedido.

O cálculo do desconto seria:

$$400 \times 0,05 = \text{R\$ } 20,00$$

Consequentemente, o valor final esperado seria:

$$400 - 20 + 30 = \text{R\$ } 410,00$$

Resultado obtido

O sistema apresenta os seguintes valores:

- **Subtotal:** R\$ 400,00
- **Desconto:** R\$ 0,00
- **Frete:** R\$ 30,00
- **Total:** R\$ 430,00

Erro identificado

A falha está no operador $>$, utilizado na condição:

$\text{qtd} > 5$

Esse operador considera somente quantidades maiores que cinco e, por isso, deixa de incluir o próprio valor 5, que corresponde ao limite estabelecido para a aplicação do desconto.

Correção realizada

A condição foi alterada de:

```
if (qtd > 5) {
```

para:

```
if (qtd >= 5) {
```

Resultado após a correção

Com a nova condição, o teste passa a apresentar:

5 >= 5 => verdadeiro

Dessa forma, o desconto de 5% é aplicado:

Desconto = R\$ 20,00

E o novo valor total passa a ser:

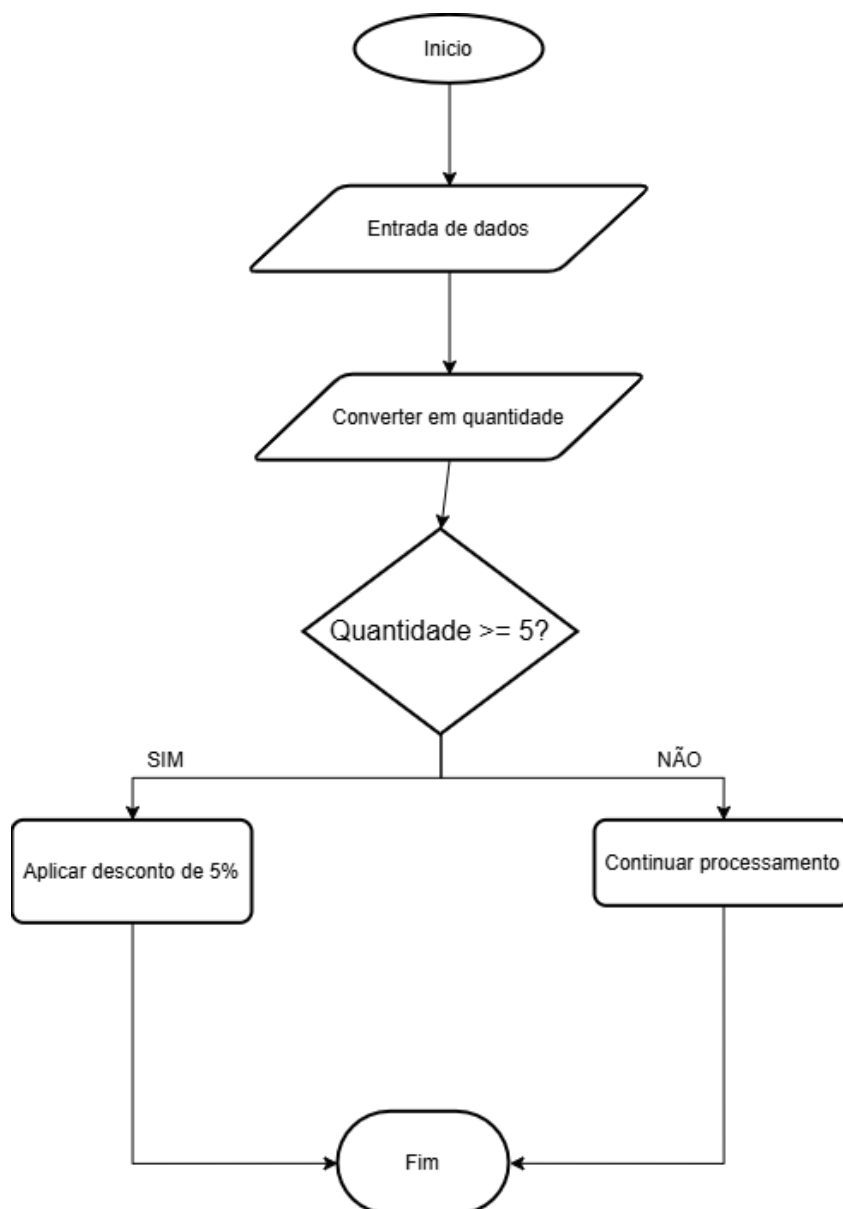
Total = R\$ 410,00

Técnica utilizada

Análise de valores-limite e rastreamento de variáveis.

A análise de valores-limite foi utilizada para verificar o comportamento do sistema exatamente no limite de cinco unidades. O rastreamento de variáveis permitiu acompanhar as alterações realizadas no subtotal, no desconto e no total durante o processamento.

Fluxograma



8. ERRO 4 – LIMITE DO PEDIDO DE ALTO VALOR

Nível: Médio

Trecho do código

```
if (total > 3000) {  
    total = total * 0.95;  
}
```

A classificação do pedido utiliza outra condição:

```
else if (total >= 3000) {  
    mensagem = "Pedido de alto valor.";  
}
```

Comportamento esperado

O limite de R\$ 3.000,00 deve ser tratado da mesma maneira em todas as regras relacionadas ao pedido de alto valor.

Como a classificação utiliza:

$\text{total} \geq 3000$

um pedido cujo valor seja exatamente R\$ 3.000,00 deve ser considerado de alto valor. Portanto, a aplicação do desconto também precisa incluir esse mesmo limite.

Dados utilizados no teste

- **Produto:** Notebook
- **Quantidade:** 1
- **Cupom:** Nenhum
- **Frete:** Retirada

Caminho percorrido

A quantidade informada é:

$\text{qtd} = 1$

A primeira validação verifica:

$1 < 0 \Rightarrow \text{falso}$

Como o valor é válido, o programa continua.

Na sequência, ocorre a verificação do estoque:

$1 \geq 5 \Rightarrow \text{falso}$

O estoque é suficiente para atender ao pedido.

O subtotal calculado é:

$$3000 \times 1 = \text{R\$ } 3.000,00$$

Como a modalidade escolhida é retirada, não há cobrança de frete:

$$\text{Retirada} = \text{R\$ } 0,00$$

Assim, o valor total antes do desconto permanece:

$$\text{Total} = \text{R\$ } 3.000,00$$

O programa então verifica a condição responsável pelo desconto:

$$3000 > 3000 \Rightarrow \text{falso}$$

Como a comparação resulta em falso, o desconto de 5% não é aplicado.

Posteriormente, o programa realiza a classificação:

$$3000 \leq 0 \Rightarrow \text{falso}$$

Em seguida:

$$3000 \geq 3000 \Rightarrow \text{verdadeiro}$$

Assim, o sistema classifica o pedido como:

Pedido de alto valor.

Resultado esperado

Como o valor de R\$ 3.000,00 corresponde ao limite utilizado para identificar pedidos de alto valor, o desconto de 5% também deveria ser aplicado.

O cálculo seria:

$$3000 \times 0,05 = \text{R\$ } 150,00$$

Consequentemente, o valor final esperado seria:

$3000 - 150 = \text{R\$ } 2.850,00$

Resultado obtido

O sistema mantém o valor do pedido em R\$ 3.000,00:

- **Subtotal:** R\$ 3.000,00
- **Desconto:** R\$ 0,00
- **Frete:** R\$ 0,00
- **Total:** R\$ 3.000,00

A mensagem apresentada é:

Pedido de alto valor.

Erro identificado

A inconsistência ocorre porque o código utiliza operadores diferentes para duas regras que consideram o mesmo limite.

Para conceder o desconto, é utilizada a condição:

`total > 3000`

Já para classificar o pedido, é utilizada:

`total >= 3000`

Com isso, um pedido de exatamente R\$ 3.000,00 é identificado como de alto valor, mas não recebe o desconto correspondente. O problema está, portanto, na diferença de tratamento do valor-limite.

Correção realizada

A condição responsável pelo desconto foi alterada de:

`if (total > 3000) {`

para:

`if (total >= 3000) {`

Resultado após a correção

Com a nova comparação:

$3000 \geq 3000 \rightarrow \text{verdadeiro}$

O desconto passa a ser aplicado:

$$3000 \times 0,95 = \text{R\$ } 2.850,00$$

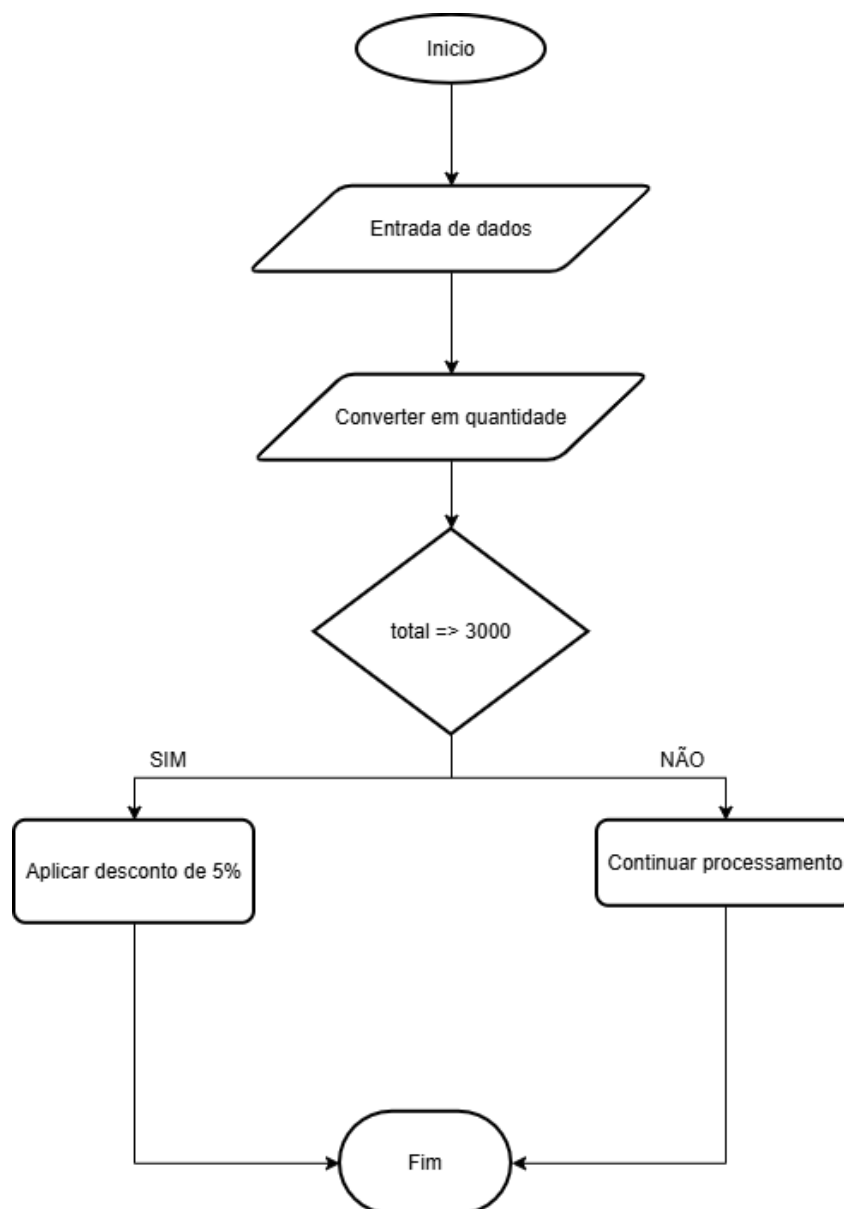
Assim, o valor do pedido após o desconto passa a ser **R\$ 2.850,00**.

Técnica utilizada

Análise de valores-limite e rastreamento de variáveis.

A análise de valores-limite foi utilizada para verificar especificamente o comportamento do sistema no valor de R\$ 3.000,00. O rastreamento de variáveis permitiu acompanhar a alteração do total durante a aplicação do desconto.

Fluxograma



9. ERRO 5 – PERDA DE ESTADO NA MENSAGEM DE ALTO VALOR

Nível: Difícil

Trecho do código

```
if (total >= 3000) {  
    total = total * 0.95;  
}
```

```
let mensagem = "Pedido calculado com sucesso.";
```

```
if (total <= 0) {  
    mensagem = "Valor do pedido inválido.";  
} else if (total >= 3000) {  
    mensagem = "Pedido de alto valor.";  
}
```

Comportamento esperado

Quando o valor inicial de um pedido atingir R\$ 3.000,00 ou mais, ele deverá ser reconhecido como um pedido de alto valor.

Para isso, a verificação do limite precisa ser realizada utilizando o valor original do pedido, antes que o desconto seja aplicado e altere a variável total.

Dados utilizados no teste

- **Produto:** Notebook
- **Quantidade:** 1
- **Preço:** R\$ 3.000,00
- **Cupom:** Nenhum
- **Frete:** Retirada
- **Valor do frete:** R\$ 0,00

Caminho percorrido

Primeiramente, o sistema calcula o subtotal do pedido:

$$3000 \times 1 = \text{R\$ } 3.000,00$$

Como a modalidade escolhida foi retirada, não existe cobrança de frete:

$$\text{Frete} = \text{R\$ } 0,00$$

Portanto, o valor inicial armazenado na variável total é:

$$\text{Total} = \text{R\$ } 3.000,00$$

O programa então verifica se o pedido atingiu o limite para aplicação do desconto:

$3000 \geq 3000 \Rightarrow \text{verdadeiro}$

Como a condição é verdadeira, o desconto é aplicado:

$3000 \times 0,95 = \text{R\$ } 2.850,00$

Após essa operação, a variável total deixa de armazenar o valor original e passa a conter:

$\text{total} = \text{R\$ } 2.850,00$

Em seguida, o sistema utiliza essa mesma variável para verificar se o pedido deve ser classificado como de alto valor:

$2850 \geq 3000 \Rightarrow \text{falso}$

Como o resultado é falso, a mensagem de pedido de alto valor não é apresentada.

Resultado esperado

O sistema deveria considerar o valor original do pedido e reconhecer que ele atingiu exatamente o limite de R\$ 3.000,00.

A mensagem esperada seria:

Pedido de alto valor.

O valor final, após a aplicação do desconto, continuaria sendo:

$\text{R\$ } 3.000,00 \times 0,95 = \text{R\$ } 2.850,00$

Resultado obtido

Após o processamento, o sistema apresenta:

Pedido calculado com sucesso.

Isso acontece mesmo tendo o pedido alcançado originalmente o limite de R\$ 3.000,00.

Erro identificado

O problema ocorre porque a variável total é modificada durante a aplicação do desconto antes de ser utilizada novamente na classificação do pedido.

Inicialmente, total representa R\$ 3.000,00, valor suficiente para caracterizar um pedido de alto valor. Porém, depois do desconto, seu conteúdo passa a ser R\$ 2.850,00.

Assim, quando a classificação é realizada, o programa verifica o valor já alterado, fazendo com que a condição deixe de ser verdadeira.

Essa situação representa uma **perda do estado original da variável** durante o fluxo de execução.

Correção realizada

Para evitar que a informação sobre o valor original seja perdida, a condição deve ser armazenada antes da aplicação do desconto:

```
const ehAltoValor = total >= 3000;
```

Depois disso, o desconto pode ser aplicado utilizando essa informação:

```
if (ehAltoValor) {  
    total = total * 0.95;  
}
```

A classificação do pedido também deve utilizar a variável booleana armazenada anteriormente:

```
if (ehAltoValor) {  
    mensagem = "Pedido de alto valor.";  
}
```

Resultado após a correção

Inicialmente, o valor do pedido é:

```
total = 3000
```

A condição é avaliada e armazenada:

ehAltoValor = 3000 >= 3000

ehAltoValor = verdadeiro

Em seguida, o desconto é aplicado:

$3000 \times 0,95 = \text{R\$ } 2.850,00$

Mesmo que a variável total seja modificada para R\$ 2.850,00, a informação de que o pedido originalmente atingiu o limite permanece armazenada em ehAltoValor.

Dessa forma, a classificação é realizada corretamente e o sistema apresenta:

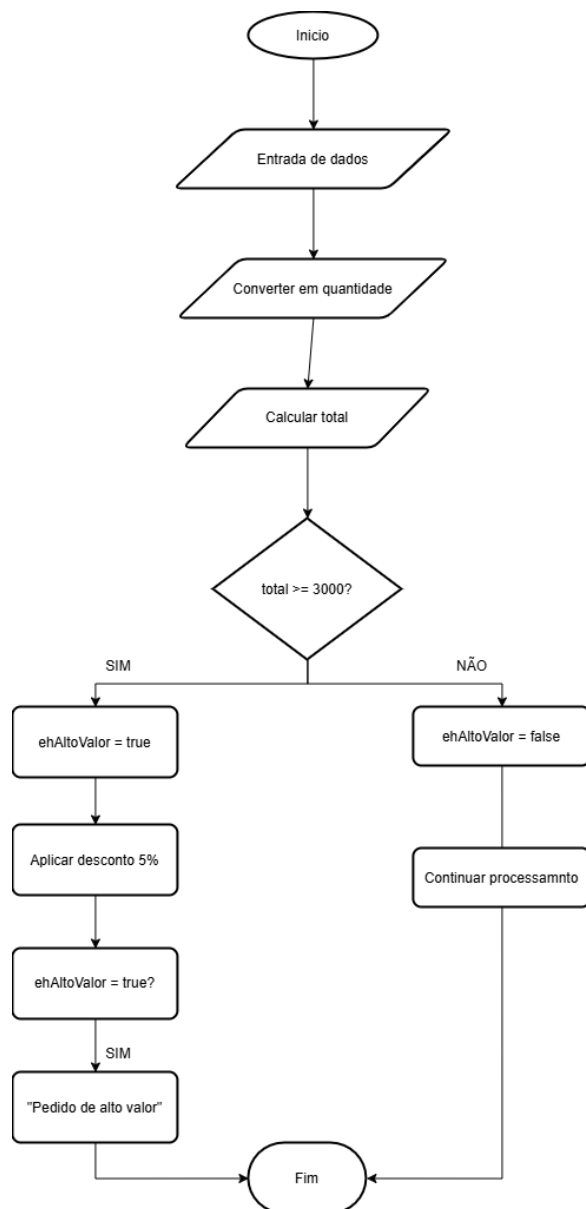
Pedido de alto valor.

Técnica utilizada

Rastreamento de variáveis e análise do fluxo de execução.

O rastreamento de variáveis permite acompanhar a alteração de total durante o processamento, enquanto a análise do fluxo de execução possibilita identificar em qual momento a informação necessária para a classificação é perdida.

Fluxograma



10. ERRO 6 – ORDEM INVERTIDA NO CÁLCULO E SOBREPOSIÇÃO DE FRETE

Nível: Difícil

Trecho do código

```
let total = subtotal - desconto + valorFrete;
```

```
if (qtd >= 5) {  
    total = total - subtotal * 0.05;
```

}

Comportamento esperado

Os descontos dos produtos devem ser calculados considerando somente o subtotal dos itens. O frete deve ser acrescentado posteriormente, mantendo-se separado dos descontos aplicados aos produtos.

A ordem esperada para o processamento é:

Subtotal => Desconto do cupom => Desconto por quantidade => Adicionar frete => Total final

Essa sequência permite que cada regra de negócio seja aplicada no momento correto e evita que o valor do frete interfira em cálculos relacionados aos produtos.

Dados utilizados no teste

- **Produto:** Mouse
- **Quantidade:** 5
- **Preço unitário:** R\$ 80,00
- **Cupom:** Nenhum
- **Frete:** Expresso
- **Valor do frete:** R\$ 60,00

Caminho percorrido

Primeiramente, é calculado o subtotal dos produtos:

$$80 \times 5 = \text{R\$ } 400,00$$

Como a quantidade é igual a cinco, o desconto previsto para essa regra corresponde a:

$$400 \times 0,05 = \text{R\$ } 20,00$$

O frete selecionado é o expresso, com valor de:

R\$ 60,00

Entretanto, o código atual adiciona o frete ao subtotal antes de aplicar o desconto:

total = 400 + 60
total = R\$ 460,00

Depois disso, o desconto por quantidade é aplicado:

$460 - 20 = \text{R\$ } 440,00$

Neste caso específico, o valor final obtido é igual ao resultado esperado. Mesmo assim, a ordem das operações não corresponde à sequência adequada das regras de negócio.

Isso acontece porque a variável total passa a conter o valor do frete antes de ser utilizada em outras decisões do sistema.

Resultado esperado

O desconto dos produtos deve ser aplicado antes que o frete seja incorporado ao valor final.

Primeiro:

Subtotal = R\$ 400,00

Depois, é aplicado o desconto por quantidade:

$400 \times 0,05 = \text{R\$ } 20,00$

O valor dos produtos após o desconto fica:

$400 - 20 = \text{R\$ } 380,00$

Somente depois o frete deve ser adicionado:

$380 + 60 = \text{R\$ } 440,00$

Portanto, o total esperado é R\$ 440,00.

Resultado obtido

Neste caso de teste, o sistema também apresenta:

R\$ 440,00

Apesar de o resultado numérico ser o mesmo, o fluxo utilizado pelo código não segue a ordem definida para as regras de negócio.

O processamento atual ocorre desta maneira:

Subtotal + Frete =>Total=>Desconto

Essa estrutura pode gerar problemas nas decisões realizadas posteriormente, principalmente nesta condição:

```
if (total >= 3000)
```

Como o frete já está incluído em total, um valor de frete elevado pode aumentar artificialmente o valor utilizado para verificar se o pedido atingiu o limite de alto valor.

Erro identificado

A falha está na sequência em que as operações são realizadas. O código mistura o valor do frete com o valor dos produtos antes de concluir os descontos.

Isso cria uma dependência inadequada entre o cálculo do frete e as regras que utilizam total para tomar decisões. Dessa forma, uma taxa de frete pode interferir em condições que deveriam considerar apenas o valor dos produtos após os descontos.

O problema pode ser caracterizado como uma inversão na precedência das regras de negócio, além de um acoplamento inadequado entre o cálculo do frete e o cálculo dos descontos.

Correção realizada

Para separar as etapas, os descontos devem ser calculados primeiro e o frete deve ser acrescentado somente ao final.

Uma estrutura adequada seria:

```
let totalProdutos = subtotal - desconto;

if (qtd >= 5) {
    totalProdutos = totalProdutos - subtotal * 0.05;
}

let total = totalProdutos + valorFrete;
```

Com essa organização, o processamento passa a seguir a sequência:

Produtos => Descontos => Valor dos produtos após descontos => Adicionar frete => Total final

Resultado após a correção

Para os dados utilizados no teste:

Subtotal = R\$ 400,00

Desconto por quantidade:

R\$ 20,00

Valor dos produtos depois do desconto:

$400 - 20 = \text{R\$ } 380,00$

Frete:

R\$ 60,00

Total final:

$380 + 60 = \text{R\$ } 440,00$

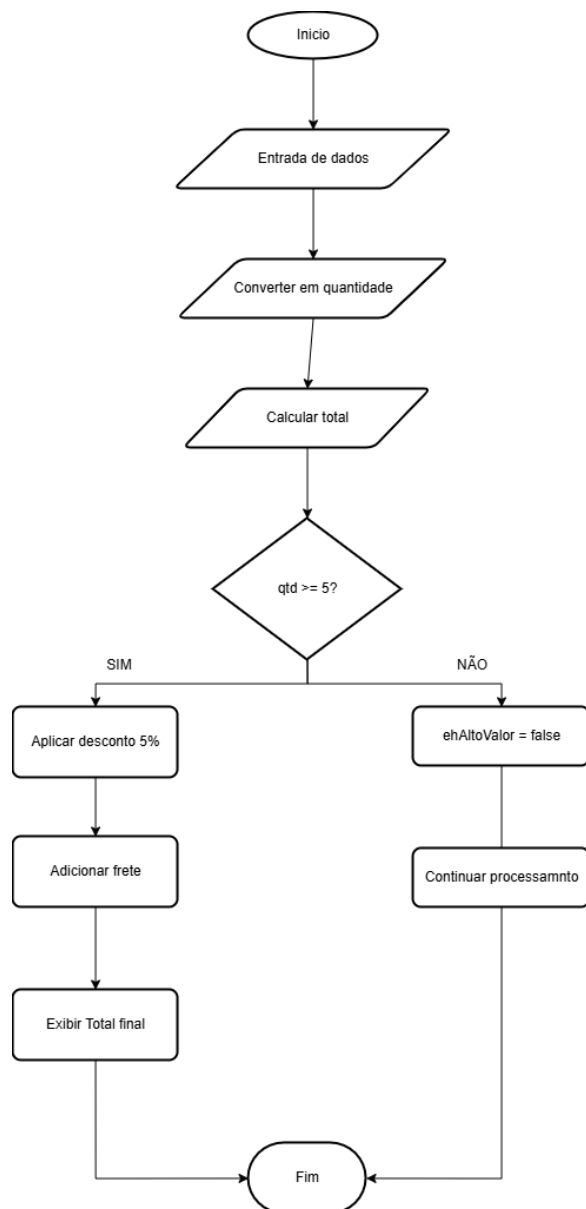
O resultado continua sendo R\$ 440,00, porém o caminho de execução passa a respeitar a ordem estabelecida para as regras de negócio e mantém o cálculo dos produtos separado do valor do frete.

Técnica utilizada

Cobertura de caminhos e rastreamento de variáveis.

A cobertura de caminhos permite analisar a sequência percorrida pelo programa, enquanto o rastreamento de variáveis possibilita acompanhar as alterações realizadas em subtotal, totalProdutos, valorFrete e total durante o processamento.

Fluxograma



11. CÓDIGO CORRIGIDO

Após a análise, as alterações realizadas foram:

```
const precos={
  notebook:3000,
  mouse:80,
  teclado:150
};
```

```
const estoque={
  notebook:5,
```

```
mouse:20,
teclado:10
};

const produto=document.getElementById("produto");
const quantidade=document.getElementById("quantidade");
const cupom=document.getElementById("cupom");
const frete=document.getElementById("frete");
const calcular=document.getElementById("calcular");
const resultado=document.getElementById("resultado");

function calcularDesconto(subtotal,codigo){
  if(codigo==="SENAI10"){
    return subtotal*0.10;
  }

  if(codigo==="SENAI20"&&subtotal>=1000){
    return subtotal*0.20;
  }

  return 0;
}

function calcularFrete(tipo,subtotal){
  if(tipo==="retirada"){
    return 0;
  }

  if(tipo==="expresso"){
    return 60;
  }

  if(subtotal>=500){
    return 0;
  }

  return 30;
}

function finalizarPedido(){
  const produtoSelecioneado=produto.value;
  const qtd=Number(quantidade.value);
  const codigo=cupom.value.trim().toUpperCase();

  if(qtd<=0){
    resultado.innerHTML="<p>Quantidade inválida.</p>";
  }
}
```

```

    return;
}

if(qtd>estoque[produtoSelecioneado]){
    resultado.innerHTML="<p>Quantidade indisponível em estoque.</p>";
    return;
}

const subtotal=precos[produtoSelecioneado]*qtd;
const desconto=calcularDesconto(subtotal,codigo);
const valorFrete=calcularFrete(frete.value,subtotal);

let totalProdutos=subtotal-desconto;

if(qtd>=5){
    totalProdutos=totalProdutos-subtotal*0.05;
}

const ehAltoValor=totalProdutos>=3000;

if(ehAltoValor){
    totalProdutos=totalProdutos*0.95;
}

const total=totalProdutos+valorFrete;

let mensagem="Pedido calculado com sucesso.";

if(total<=0){
    mensagem="Valor do pedido inválido.";
}else if(ehAltoValor){
    mensagem="Pedido de alto valor.";
}

resultado.innerHTML=`
    <p>${mensagem}</p>
    <p>Subtotal: R$ ${subtotal.toFixed(2)}</p>
    <p>Desconto: R$ ${(subtotal-totalProdutos).toFixed(2)}</p>
    <p>Frete: R$ ${valorFrete.toFixed(2)}</p>
    <p class="total">Total: R$ ${total.toFixed(2)}</p>
`;
}

calcular.addEventListener("click",finalizarPedido);

```

12. NOVOS TESTES APÓS AS CORREÇÕES

Erro	Análise realizada	Técnica utilizada	Resultado
Erro 1 — Validação da quantidade	Foi analisado o limite mínimo da quantidade. O valor 0 demonstrou que a condição $qtd < 0$ permitia uma quantidade que deveria ser inválida	Análise de valores-limite	A condição foi alterada para $qtd \leq 0$, logo, fazendo com que zero seja rejeitado.
Erro 2 — Limite do estoque	Foi testada uma quantidade exatamente igual ao estoque disponível. A condição $qtd \geq \text{estoque}$ rejeitava uma quantidade que ainda estava disponível.	Análise de valores-limite	A condição foi alterada para $qtd > \text{estoque}$, assim permitindo a compra da quantidade exata disponível.
Erro 3 — Desconto por quantidade	Foi analisado o limite de cinco unidades. A condição $qtd > 5$ não aplicava o desconto quando a quantidade era exatamente 5	Análise de valores-limite e rastreamento de variáveis	A condição foi alterada para $qtd \geq 5$, permitindo o desconto de 5% a partir de cinco unidades.
Erro 4 — Limite do pedido de alto valor	Teve análise da diferença entre $\text{total} > 3000$ e $\text{total} \geq 3000$. O valor exato de R\$ 3.000,00 recebia tratamentos diferentes.	Análise de valores-limite e rastreamento de variáveis	A condição de aplicação do desconto foi alterada para $\text{total} \geq 3000$.
Erro 5 — Perda de estado na mensagem de alto valor	Teve o acompanhamento do valor da variável total antes e depois da aplicação do desconto. Após o desconto, R\$ 3.000,00 passava para R\$ 2.850,00, fazendo a classificação posterior retornar falso.	Rastreamento de variáveis e análise de fluxo de execução	Foi criada a variável <code>ehAltoValor</code> antes da alteração de total, preservando a informação de que o pedido atingiu o limite.

13. COMPARAÇÃO ANTES E DEPOIS

Problema	Antes da correção	Depois da correção
Quantidade 0	Era aceita pelo sistema	É rejeitada como quantidade inválida
Quantidade igual ao estoque	Era rejeitada	É permitida quando corresponde exatamente ao estoque disponível
Quantidade 5	Não recebia desconto	Recebe desconto de 5%
Total de R\$ 3.000,00	Não recebia o desconto de alto valor	Recebe desconto de 5%

14. ANÁLISE DOS RESULTADOS

A execução dos casos de teste mostrou como a análise da estrutura interna do código pode revelar problemas que nem sempre ficam evidentes apenas observando o resultado final do sistema.

No primeiro teste, foi aplicada a análise de valores-limite sobre a quantidade mínima permitida. A utilização do valor zero mostrou que a condição implementada não estava de acordo com o limite estabelecido no campo de entrada, que determina quantidade mínima igual a 1.

No segundo teste, também foi analisado um valor localizado exatamente na fronteira da regra. Ao utilizar uma quantidade igual ao estoque disponível, foi possível identificar que o operador $\geq$ fazia com que o sistema recusasse uma quantidade que poderia ser solicitada normalmente.

O terceiro teste avaliou o limite utilizado para conceder o desconto por quantidade. A condição $qtd > 5$ considerava somente valores superiores a cinco e, consequentemente, não aplicava o desconto quando a quantidade era exatamente 5. Para atender à regra estabelecida, a condição foi alterada para $qtd \geq 5$.

No quarto teste, foram comparadas duas condições que utilizavam o limite de R\$ 3.000,00. Enquanto o desconto era aplicado somente quando $total > 3000$, a classificação como pedido de alto valor utilizava $total \geq 3000$. Essa diferença fazia com que um pedido exatamente igual a R\$ 3.000,00 fosse tratado de maneira diferente em cada etapa.

Durante os testes, também foi possível acompanhar o comportamento das principais variáveis utilizadas no processamento:

- qtd
- subtotal
- desconto
- valorFrete

- total
- ehAltoValor

O rastreamento dessas informações permitiu observar não somente qual resultado era produzido, mas também as etapas percorridas pelo programa até chegar a esse resultado. Dessa forma, foi possível identificar em que momento o comportamento do sistema se afastava da regra esperada.

As condições relacionadas aos cupons de desconto e ao cálculo do frete também foram verificadas. Entretanto, como as regras apresentadas no enunciado não estabeleciam limites ou comportamentos diferentes daqueles implementados no código, essas condições não foram consideradas erros nesta análise.

15. COLCUSÃO

A realização desta atividade possibilitou estudar o funcionamento de um sistema de pedidos por meio de técnicas de teste de caixa branca, permitindo analisar tanto os resultados apresentados quanto as etapas internas responsáveis por esses resultados.

A leitura do código possibilitou identificar as principais estruturas condicionais que determinam os diferentes caminhos de execução. Além disso, a utilização de valores próximos aos limites das regras ajudou a encontrar situações em que o comportamento do sistema não correspondia ao esperado.

Os testes mostraram que pequenas diferenças entre operadores relacionais, como `<`, `<=`, `>` e `>=`, podem alterar o resultado de uma condição e, consequentemente, modificar o funcionamento do programa. Ao todo, foram encontrados seis problemas de lógica envolvendo a validação da quantidade, a verificação do estoque, o desconto por quantidade, o limite para pedidos de alto valor, a alteração do valor utilizado para classificar o pedido e a ordem de aplicação dos descontos e do frete.

No quinto erro, o rastreamento das variáveis foi essencial para identificar o problema. A variável `total` era modificada quando o desconto era aplicado e, depois disso, era utilizada novamente para verificar se o pedido deveria ser considerado de alto valor. Como o valor já havia sido reduzido, o sistema não conseguia mais identificar que o pedido havia atingido originalmente R\$ 3.000,00. A utilização da variável `ehAltoValor` resolveu esse problema ao preservar o resultado da condição antes da alteração de `total`.

O sexto erro mostrou a importância da ordem em que as operações são executadas. Foi identificado que o frete era somado ao total antes da aplicação do

desconto por quantidade. A reorganização desse processo separou o cálculo dos descontos do cálculo do frete, deixando cada etapa responsável por sua própria regra e tornando o fluxo mais adequado ao funcionamento esperado do sistema.

Depois das alterações, os casos de teste foram executados novamente para verificar se as falhas identificadas haviam sido corrigidas e se os resultados obtidos estavam de acordo com os comportamentos esperados.

O acompanhamento das variáveis `qtd`, `subtotal`, `desconto`, `valorFrete`, `total` e `ehAltoValor` também contribuiu para compreender o processamento de cada pedido. Com esse rastreamento, foi possível identificar exatamente em qual etapa os valores deixavam de seguir a lógica definida.

Portanto, a atividade demonstrou que o teste de caixa branca não se limita à verificação do resultado final de uma aplicação. A análise das condições, dos valores-limite, dos caminhos de execução e das alterações sofridas pelas variáveis permite compreender o funcionamento interno do programa e encontrar falhas de lógica que poderiam não ser percebidas apenas pela comparação entre entradas e saídas.